Industrial Internship Report on

Human Resource Management System (HRMS)

Prepared by

Md Aariz Farooqui

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).

This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time.

My project was Human Resource Management System (HRMS)

This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

Preface

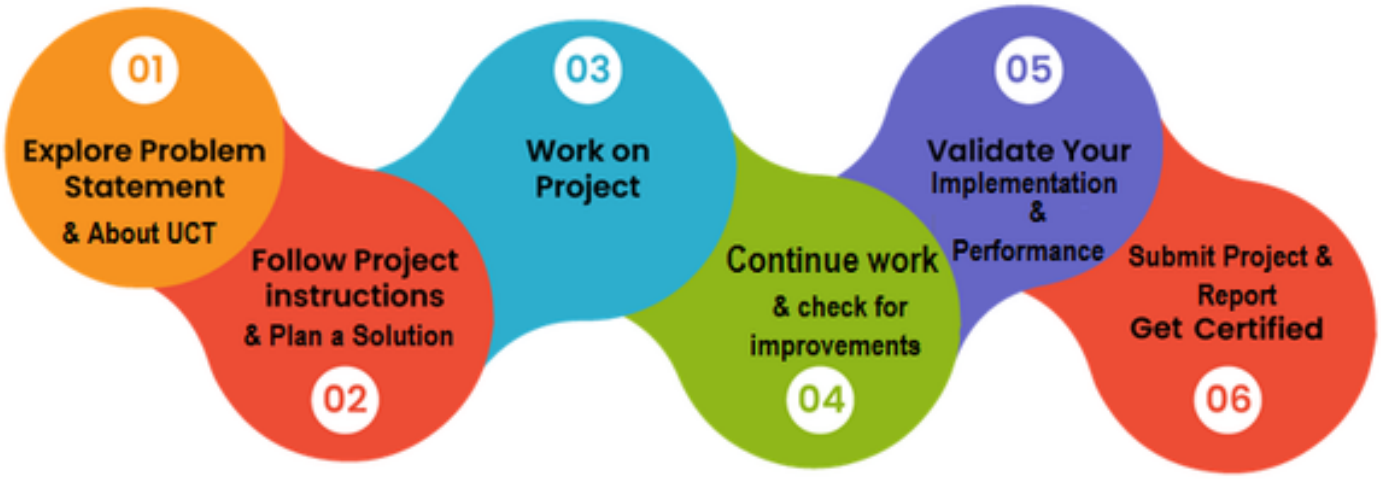
Summary of the whole 6 weeks' work.

About need of relevant Internship in career development.

Brief about Your project/problem statement.

Opportunity given by USC/UCT.

How Program was planned



Your Learnings and overall experience.

Thank to all (with names), who have helped you directly or indirectly.

Your message to your juniors and peers.

Introduction

About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.

i. UCT IoT Platform (

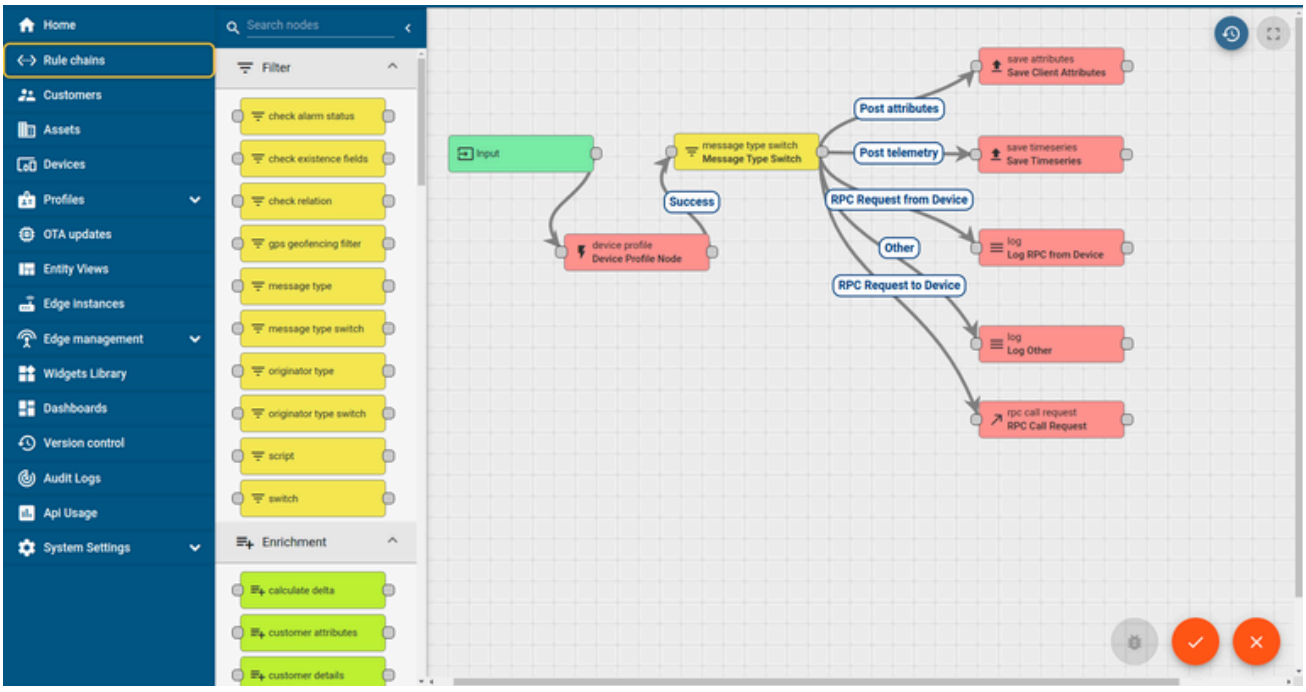
uct Insight

i.)

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to• Build Your own dashboard• Analytics and Reporting• Alert and Notification• Integration with third party application(Power BI, SAP, ERP)• Rule Engine



i. **Smart Factory Platform (**

FACTORY WATCH

i.)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.

i.

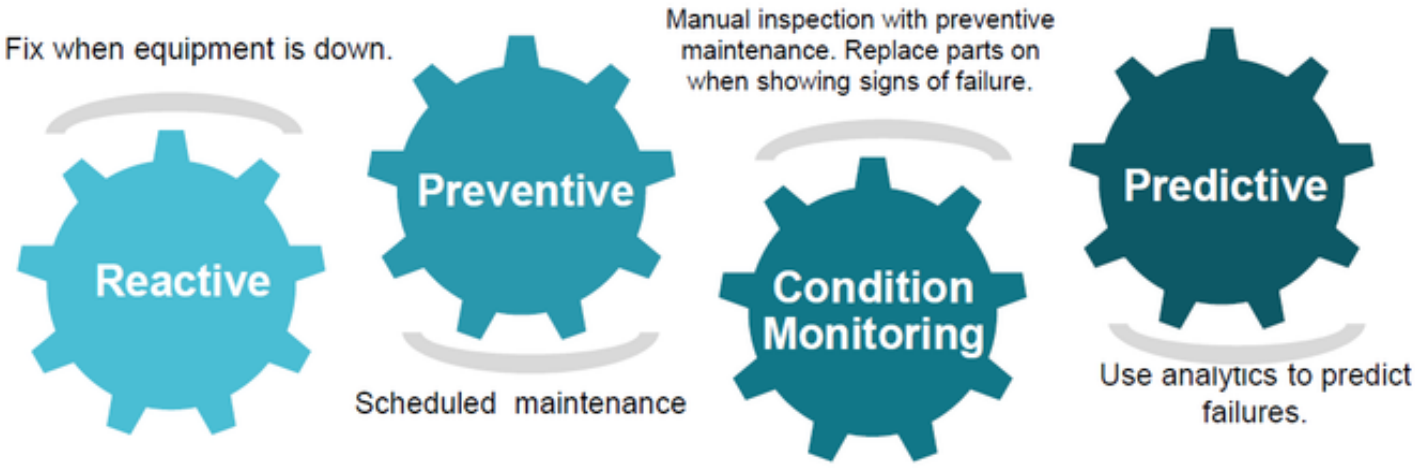
LoRaWAN™

i. based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

i. Predictive Maintenance

UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.

The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

-

Objectives of this Internship program

The objective for this internship program was to

☞ get practical experience of working in the industry.

☞ to solve real world problems.

☞ to have improved job prospects.

☞ to have Improved understanding of our field and its applications.

☞ to have Personal growth like better communication and problem solving.

Reference

- Oracle Java 11 Documentation — https://docs.oracle.com/en/java/javase/11/
- JUnit 5 User Guide — https://junit.org/junit5/docs/current/user-guide/
- Maven Getting Started Guide — https://maven.apache.org/guides/
- GitHub — https://github.com
- upskill Campus — https://www.upskillcampus.com/

Glossary

Term / Acronym	Description
HRMS	Human Resource Management System
UCT	UniConverge Technologies Pvt Ltd
USC	upskill Campus
OOP	Object-Oriented Programming
CRUD	Create, Read, Update, Delete
SHA-256	Secure Hash Algorithm — used for password hashing
CSV / TXT	Flat text file used for data persistence
JUnit	Java unit testing framework
Maven	Java build and dependency management tool

Problem Statement

Organisations of all sizes face significant challenges in managing their human resources manually. Tracking employee data in spreadsheets, recording attendance on paper registers, and handling leave requests through informal email chains leads to data inconsistency, slow processing, and poor visibility into workforce status.

The problem statement assigned by UCT was to design and develop a console-based Human Resource Management System (HRMS) using Core Java that addresses the following key pain points:

- No centralised system to store and retrieve employee records efficiently.
- Manual and error-prone attendance tracking with no audit trail.
- Leave requests managed through informal channels with no approval workflow.
- No quick way to search for employees by name, department, or ID.
- No automated report generation to give HR administrators visibility into workforce patterns.
- No access control — anyone could view or modify records without authentication.

The solution was required to meet the following functional requirements:

#	Feature	Requirement
1	User Authentication	Secure login for HR administrators with hashed passwords
2	Employee Management	Add, view, update, and delete employee records
3	Attendance Tracking	Mark employees as Present, Absent, or On Leave per date
4	Leave Management	Submit, approve, and reject leave requests with auto-attendance update
5	Employee Search	Search by name, employee ID, or department
6	Report Generation	Generate and save employee, attendance, and leave reports
7	Data Persistence	Store all data in flat text files — no external database
8	Error Handling	Graceful handling of all invalid inputs and exceptions

Existing and Proposed solution

4.1 Existing Solutions and Their Limitations

Several HR management solutions exist in the market today, including enterprise platforms such as SAP HCM, Workday, and BambooHR, as well as open-source tools. However, these come with significant limitations in the context of this use case:

- Enterprise solutions like SAP and Workday require expensive licences, dedicated servers, and IT support teams — not suitable for small organisations.
- Most existing tools depend on relational databases (MySQL, PostgreSQL), requiring database setup, configuration, and maintenance.
- Cloud-based HR tools require internet connectivity and raise data privacy concerns for organisations with sensitive employee data.
- Academic-level projects typically use databases but lack clean architecture, error handling, and proper authentication.
- None of the lightweight solutions available offer all six required features (auth, CRUD, attendance, leave, search, reports) in a single self-contained Java application.

4.2 Proposed Solution

The proposed solution is a self-contained, console-based HRMS built entirely in Core Java with no external dependencies (no database, no web server, no third-party libraries). Key value additions of this solution include:

- Zero-dependency deployment — runs with just Java 11+ installed, on any OS.
- Flat file persistence — all data stored in human-readable pipe-delimited .txt files in a data/ folder, auto-created on first run.
- SHA-256 password hashing — secure authentication without storing plaintext passwords.
- Auto-integration between modules — approving a leave request automatically marks attendance as ON_LEAVE for the entire date range.
- Formatted report files — reports saved as .txt files with timestamps to a reports/ folder for record-keeping.
- Layered architecture — clean separation of UI, business logic, data access, and models.

Code submission (Github link) :

https://github.com/Aariz171/upskillCampus/tree/bcbe59b115b8466f4f22a8f38e1d8e0ad6e2e6e2/HRMS

Report submission (Github link) : GitHub - Aariz171/upskillCampus

Proposed Design/ Model

Given more details about design flow of your solution. This is applicable for all domains. DS/ML Students can cover it after they have their algorithm implementation. There is always a start, intermediate stages and then final outcome.

High Level Architecture (if applicable)

The HRMS follows a 4-layer architecture ensuring clean separation of concerns:

Layer	Classes	Responsibility
UI Layer	EmployeeMenu, AttendanceMenu, LeaveMenu, SearchMenu, ReportMenu, MainMenu	Console input/output, menu navigation, user interaction
Service Layer	EmployeeService, AttendanceService, LeaveService, ReportService	Business logic, validation, cross-module coordination
Repository Layer	EmployeeRepository, AttendanceRepository, LeaveRepository	File read/write operations, data access
Model Layer	Employee, Admin, Attendance, LeaveRequest + enums	Data structures, serialisation to/from file strings
Utils Layer	FileHandler, HashUtils, ConsoleUtils, HRMSException, ValidationUtils	Cross-cutting concerns: I/O, hashing, validation, error handling

Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

Low Level Design (if applicable)

5.2.1 Data Flow

User Input (Console) → UI Menu → Service (validate + business logic) → Repository (read/write file) → data/*.txt

5.2.2 File Storage Schema

All data is stored in pipe-delimited plain text files:

File	Format	Example
data/employees.txt	ID|Name|Designation|Dept|Email|Phone|JoinDate	EMP001|Alice|Sr Dev|Engineering|...|...|2024-01-15
data/attendance.txt	EmpID|Date|Status	EMP001|2026-04-28|PRESENT
data/leaves.txt	LeaveID|EmpID|Type|Start|End|Status|Reason	LV123|EMP001|SICK|2026-05-05|2026-05-07|APPROVED|Fever
data/admins.txt	Username|SHA256Hash	admin|240be518fa...

5.2.3 Module Interaction

The key cross-module integration is between the Leave and Attendance modules. When a leave request is approved via LeaveService.approveLeave(), it internally calls AttendanceService.markRangeAsOnLeave() to automatically set ON_LEAVE status for every date in the approved leave range. This ensures attendance records stay consistent without requiring manual updates.

5.3 Key Interfaces and Classes

The project contains 27 Java source files organized across 6 packages:

Package	Classes
com.hrms.main	Main.java
com.hrms.auth	AuthService.java
com.hrms.models	Employee, Admin, Attendance, LeaveRequest, AttendanceStatus, LeaveType, LeaveStatus
com.hrms.repositories	EmployeeRepository, AttendanceRepository, LeaveRepository
com.hrms.services	EmployeeService, AttendanceService, LeaveService, ReportService
com.hrms.ui	MainMenu, EmployeeMenu, AttendanceMenu, LeaveMenu, SearchMenu, ReportMenu
com.hrms.utils	ConsoleUtils, FileHandler, HashUtils, HRMSException, ValidationUtils

Performance Test

Test Plan / Test Cases

The project includes 40 JUnit 5 unit tests across 4 test classes covering all core service and utility functions:

Test Class	Tests	Coverage
EmployeeServiceTest	13 tests	Add, view, update, delete, search, duplicate check, not-found
AttendanceServiceTest	7 tests	Mark attendance, invalid employee, invalid date, range marking
LeaveServiceTest	8 tests	Submit, approve, reject, invalid dates, already-processed guard
ValidationUtilsTest	12 tests	Email, phone, date, date range, employee ID, empty field validation

Test Procedure

Tests were executed using Maven with the following command:

```
mvn clean test
```

Each test class uses @BeforeAll to initialise data files in a test directory and @AfterAll to clean up, ensuring tests are isolated and repeatable. Tests are ordered using @TestMethodOrder(MethodOrderer.OrderAnnotation.class) to ensure dependent tests (e.g. approve before reject) run in sequence.

Performance Outcome

Metric	Result
Total Tests Run	40
Tests Passed	40 (100%)
Tests Failed	0
Build Status	SUCCESS
Total Execution Time	~3.1 seconds
File I/O Performance	< 50ms per read/write operation
Login Authentication	< 10ms (SHA-256 hash + file lookup)

Identified constraints and mitigation:

- File size growth: As records accumulate, file read times increase linearly. Mitigation — for large scale, replace FileHandler with a lightweight database (SQLite).
- Concurrent access: Flat file storage does not support concurrent writes. Mitigation — acceptable for single-admin console use; upgrade to a database for multi-user scenarios.
- No UI framework: Console-only limits usability. Mitigation — the architecture is designed so UI layer can be replaced with a web front-end without changing services or repositories.

My learnings

This 6-week internship was a transformative experience that bridged the gap between academic knowledge and real-world software development. Below is a summary of the key technical and professional learnings:

7.1 Technical Learnings

- Layered Architecture: Learned to design software in distinct layers (UI, Service, Repository, Model, Utils) — making code modular, testable, and maintainable.
- Object-Oriented Design: Applied OOP principles including encapsulation, abstraction, and single responsibility across 27 Java classes.
- File I/O in Java: Gained hands-on experience with BufferedReader, BufferedWriter, and FileWriter for persistent data storage without a database.
- Security: Implemented SHA-256 password hashing using Java's MessageDigest — understanding why plaintext passwords must never be stored.
- Exception Handling: Designed a custom HRMSException hierarchy and applied try-catch at service and UI layers for graceful error handling.
- Enums and Java Collections: Used enums (AttendanceStatus, LeaveType, LeaveStatus) and ArrayList/LinkedHashMap for clean, type-safe data modelling.
- Unit Testing with JUnit 5: Wrote 40 unit tests using @Test, @BeforeAll, @AfterAll, assertThrows, assertEquals — understanding TDD principles.
- Build Tools: Used Maven (pom.xml) to manage project structure, compile, test, and package the application.
- Version Control: Used Git with feature branches, meaningful commits, and Pull Requests to manage the project on GitHub.

7.2 Professional Learnings

- Project Planning: Breaking a 4-week project into weekly milestones made delivery manageable and kept progress visible.
- Documentation: Maintaining a README, inline Javadoc comments, and this formal report taught the importance of clear technical writing.
- Problem Solving: Encountering and debugging errors (class-path issues, file encoding, null pointer exceptions) built confidence in independent troubleshooting.
- Structured Thinking: Designing the system before writing code — starting with UML class diagrams and data schemas — significantly reduced rework.

Future work scope

Due to the 6-week time constraint, several enhancements could not be incorporated but represent promising future directions for this project:

#	Enhancement	Description
1	Database Integration	Replace flat file storage with SQLite or MySQL for better scalability and concurrent access support.
2	Web / GUI Interface	Build a JavaFX desktop GUI or a Spring Boot REST API with a React front-end on top of the existing service layer.
3	Payroll Module	Add salary calculation based on attendance records, leave deductions, and employee grade.
4	Multi-Admin Support	Allow multiple admin accounts with role-based access control (RBAC) — e.g. HR Manager vs Viewer roles.
5	Email Notifications	Send email alerts to employees when their leave is approved or rejected using JavaMail API.
6	Export to PDF / Excel	Use Apache POI or iText to export reports as .xlsx or .pdf files instead of plain text.
7	Employee Self-Service	Allow employees to log in and view their own attendance and leave history through a separate employee portal.
8	Leave Balance Tracking	Track annual leave entitlements per employee and enforce limits when submitting leave requests.

The modular layered architecture of this project was specifically designed to facilitate these upgrades. The Service and Repository layers can remain untouched while the UI layer is replaced with a web interface, or the Repository layer alone can be swapped from file-based to database-backed storage.

Appendix — 4-Week Development Timeline

Week	Focus	Deliverables
Week 1	Foundation & Employee CRUD	Auth (SHA-256), Employee model, FileHandler, EmployeeService, EmployeeMenu, data persistence to .txt files
Week 2	Attendance & Leave	Attendance marking, Leave submission/approval/rejection, auto-attendance update on leave approval
Week 3	Search, Reports & Validation	Employee search (name/ID/dept), report generation saved to reports/, ValidationUtils, full error handling
Week 4	Testing & Documentation	40 JUnit 5 tests, Maven pom.xml, README, run scripts (run.bat / run.sh), GitHub release v1.0.0